



Transactions

Isolation

Isolation is a property that determines, among other things, **how and when** the operations performed concurrently by some transactions become visible to another transaction.

Most DBMSs provide **isolation levels** that aim to prevent three undesirable phenomena in concurrent execution: **dirty reads, non-repeatable reads (fuzzy reads), and phantom reads (phantom rows)**.

The **appropriate** isolation level for each application depends on its data integrity requirements and on the overhead introduced by each isolation level.

The ANSI/ISO SQL standard defines isolation levels:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable

The specific mechanisms used to achieve them depend on the DBMS.

Concurrency Control

To guarantee the specified isolation level, DBMSs use **locking mechanisms** or **multi-version concurrency control (MVCC)**.

The following is a simplified model of behavior based on locking or versioning. The specific mechanisms depend on the DBMS and its implementation.

Lock-Based Concurrency Control (Simplified Scheme)

READ UNCOMMITTED (RU) Isolation level

Operation	Lock	Lock acquired	Lock released
SELECT	None	Not applicable	Not applicable
UPDATE/DELETE/INSERT	X-lock	Modifying / inserting row	End of transaction (Commit / Rollback)

READ COMMITTED (RC) Isolation level

Operation	Lock	Lock acquired	Lock released
SELECT	S-lock over rows read	When reading each row	After reading the row
UPDATE/DELETE/INSERT	X-lock on modified rows	When modifying row	End txn

REPEATABLE READ (RR) Isolation level

Operation	Lock	Lock acquired	Lock released
SELECT	S-lock on read rows	By reading each row that meets condition	End txn
UPDATE/DELETE/INSERT	X-lock on modified rows	When modifying row	End txn

SERIALIZABLE (Ser) Isolation level

Operation	Lock	Lock acquired	Lock released
SELECT	S-lock on read rows and ranges	When reading rows or ranges	End txn
UPDATE/DELETE/INSERT	X-lock on modified rows	When modifying rows/ranges	End txn

Optimistic approach or Multi-Version Concurrency Control (Oracle)

READ UNCOMMITTED (RU) Isolation level

Not supported by Oracle.

READ COMMITTED (RC) Isolation level

When data version is obtained	Which data version is visible	Concurrency control / update conflicts
At the beginning of each SQL statement	Versions committed before the statement began	Other transactions did not commit updates or deletes on rows that this transaction updates (prevents lost-update)

REPEATABLE READ (RR) Isolation level

Not directly implemented by Oracle

When data version is obtained	Which data version is visible	Concurrency control / update conflicts
At the beginning of the transaction	Versions committed before the start of the transaction	Same as RC; however, rows read by this txn cannot have been modified or deleted by other txns since the start of this

SERIALIZABLE (Ser) Isolation level

When data version is obtained	Which data version is visible	Concurrency control / update conflicts
At the beginning of the transaction	Versions committed before the start of the transaction	Same as RR; No new rows become visible that would have matched a query executed during the txn

Oracle and HSQLDB

The two DBMS that we are going to use in this practice have the following characteristics in terms of concurrency control:

By default, they both work in **autocommit**, i.e. each sql statement is a transaction on its own and is automatically committed if it ends well. Therefore, in both cases you will have to disable Autocommit mode in both.

Oracle

Implement optimistic concurrency control (MVCC). Supports READ COMMITTED and SERIALIZABLE isolation levels as defined in the standard (https://docs.oracle.com/cd/B10501_01/server.920/a96524/c21cnsis.htm#2570).

HSQLDB

It supports different concurrency control mechanisms (<https://hsqldb.org/doc/2.0/guide/sessions-chapt.html>). By default, two-phase lock (2PL), but also multiversion currency control (MVCC).

In both cases, the standard indicates that the database engine returns a higher level of isolation than the requested one in case the requested one is not implemented. Therefore, HSQLDB promotes READ UNCOMMITTED to READ COMMITTED and REPEATABLE READ to SERIALIZABLE (in the latest version checked, 2.7.2)..

Aims

- Understand and strengthen the concepts of transaction, concurrence and isolation.
- To know the possibilities of configuring a DBMS depending on the degree of concurrency desired.
- Know the different degrees of isolation of Oracle and HSQLDB.
- To achieve this, the following cases must be tested in the two DBMS indicated and with two levels of isolation:
 - **HSQL (LOCKS) con READ COMMITTED and SERIALIZABLE**
 - **HSQL (MVCC) con READ COMMITTED and SERIALIZABLE**
 - **Oracle con READ COMMITTED and SERIALIZABLE**



Environment

Download the database "Database (hsqldb) for Travel Agency" from the Virtual Campus. Boot it in server mode. Open two sessions with the database and disable **Autocommit mode** on both.

Run SQL Developer and connect to Oracle with your username and password. Create the following table (a script is available)

London	450
Paris	320
Rome	280

Open two work sessions with **autocommit mode disabled**.

Instructions for Setting the Concurrency Control Mechanism in HSQLDB

SET DATABASE TRANSACTION CONTROL LOCKS

SET DATABASE TRANSACTION CONTROL MVCC

Instructions for changing the isolation level

To switch between different levels of isolation, the SQL statement is

SET TRANSACTION ISOLATION LEVEL XXXX

Remember to execute commit or rollback first. If you need to know the current isolation level in HSQLDB you can run

CALL ISOLATION_LEVEL()

Transactions to be executed

For each of the following scripts, it notes the results of each read operation. Are any processes crashing during execution? Which one? When? Why? What anomalies occur in each run?

Important: Make sure to rewrite the original values to the tables before each run.

A. Dirty read

A.1 Script 1

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX	SET TRANSACTION ISOLATION LEVEL XXXXXXXX
SELECT * from Ttrips	UPDATE Ttrips SET price = price + 10
	Rollback

A.2 Script 2

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX	SET TRANSACTION ISOLATION LEVEL XXXXXXXX
SELECT * from Ttrips	UPDATE Ttrips SET price = price + 10
SELECT * from Ttrips	Commit/Rollback



B. Unrepeteable read

B.1 Script 1

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT price FROM Ttrips WHERE destination = 'Paris' SELECT price FROM Ttrips WHERE destination = 'Paris' Commit SELECT price FROM Ttrips WHERE destination = 'Paris'	SET TRANSACTION ISOLATION LEVEL XXXXXXXX UPDATE Ttrips SET price = 350 WHERE destination = 'Paris' Commit

B.2 Script 2

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT * FROM Ttrips WHERE destination = 'Rome' SELECT * FROM Ttrips WHERE destination = 'Rome'; UPDATE Ttrips SET price=price+10 SELECT * FROM Ttrips	SET TRANSACTION ISOLATION LEVEL XXXXXXXX UPDATE Ttrips SET price = price + 50 WHERE destination = 'Rome' Commit

Phantom read

TRANSACTION 1 (Ej: Travel agency)	TRANSACTION 2 (Ej: Booking center)
SET TRANSACTION ISOLATION LEVEL XXXXXXXX SELECT * FROM Ttrips WHERE price BETWEEN 250 AND 350 SELECT * FROM Ttrips WHERE price BETWEEN 250 AND 350	SET TRANSACTION ISOLATION LEVEL XXXXXXXX INSERT INTO Ttrips VALUES ('Madrid',260) Commit